

PS-LAB7-289810

Zadanie 1: Napisz skrypt PowerShell, który monitoruje określony folder i automatycznie przenosi nowo dodane pliki .txt do innej lokalizacji.

a) Jeżeli folder docelowy nie istnieje, skrypt musi go utworzyć.

b) Skrypt powinien działać ciągle, aż do jego ręcznego wyłączenia.

```
$doclowa_path = Read-Host -Prompt "Podaj sciezke do folderu docelowego: "
$monitor_path = Read-Host -Prompt "Podaj sciezke do folderu ktory chcemy
monitorowac: "

#Test-Path ~ sprawdza czy ścieżka istnieje
if (!(Test-Path $doclowa_path)) {
    #New-Item tworzy nowy item, -ItemType Directory ~ jest katalogiem ~ -Path
    $doclowa_path o danej ścieżce co tworzy nam nowy katalog
    New-Item -ItemType Directory -Path $doclowa_path | Out-Null
    #out-null ~ usuwa nam odpowiedz new-item zeby nic sie nie wyswietlalo na
    konsoli
}
if (!(Test-Path $monitor_path)) {
    New-Item -ItemType Directory -Path $monitor_path | Out-Null
}

Write-Host "Mozna wyjsc z programu za pomoca ctrl+C"
#nieskończona petla
while ($true) {
    #Get-ChildItem ~wlistuje czy istnieją pliki/katalogi w danej ścieżce | -Path
    $monitor_path ~ ścieżka która sprawdzamy | -File ~ muszą to być pliki
    foreach ($file in Get-ChildItem -Path $monitor_path -File){
        # dostajemy plik jako $file
        #sprawdzamy czy coś się nie powiodło
        try {
            #Join-Path ~łączy dwie ścieżki ze sobą | $file.Name ~ bierzemy tylko
            nazwe pliku
            $destination = Join-Path $doclowa_path $file.Name
            #$file.fullname ~ ścieżka absolutna do pliku | Move-Item ~ przenosi
            plik o ścieżce podaną w -Path do wartości podanej w -Destination
            Move-Item -Path $file.FullName -Destination $destination -ErrorAction
```

Stop

```
        #-ErrorAction Stop ~ zatrzymuje i wyrzuca error przy jakiś błędach
        podczas przenoszenia plików
    } catch {
        Write-Host "coś poszło nie tak podczas przenoszenia pliku"
    }
}
#usypiamy program na pół sekundy bo nie musi ciągle sprawdzać
Start-Sleep 0.5
}
```

Zadanie 2: Napisz skrypt w PowerShell który:

a) Obliczy sumę kontrolną pliku (MD5 lub SHA256).

b) Wyśle zapytanie do API VirusTotal

c) Zinterpretuje odpowiedź API i wyświetli informację, czy plik jest bezpieczny, czy nie.

Sprawdzić, czy wyniki są zgodne z oczekiwaniami - pobrać plik EICAR oraz utworzyć nowy plik testowy. Każdy etap skryptu powinien być opatrzony komentarzem.

#funkcja która zabiera ścieżkę do pliku i zwraca hash pliku

```
function get_hash {
    #pozwala na wprowadzanie do funkcji wartości z poza funkcji
    param (
        [String]$file_path
    )
    #Get-FileHash ~ zwraca hash z pliku o ścieżce podanej w -Path | -Algorithm
    tutaj wybierzmy algorytm jaki chcemy użyć
    return (Get-FileHash -Path $file_path -Algorithm MD5).Hash
}
```

#funkcja która zabiera ścieżkę do pliku i api_key do virus_total, by wrócić raport na temat pliku

```
function get_raport {
    param (
        [String]$file_path,
        [String]$api_key
    )

    #wysyłamy nasz plik za pomocą curl POST i odbiermy infoamcje na temat naszego
    skanu

    #curl wysła zapytania do jakiejś domeny i dostaje od niej infoamcje
    # --silent ~ nie pokazuje wyników | -X wysła zapytanie(request) | POST ~
```

wysyłamy coś do domeny | -H dodajemy jakąś wartość do header | -F (--form) ~ umożliwia wystanie form data zazwyczaj w formie multipart/form-data | W from data wystamy nasz plik

```
$response = curl.exe --silent -X POST "https://www.virustotal.com/api/v3/files"
-H "x-apikey: $api_key" -F "file=@$file_path" | ConvertFrom-Json
#ConvertFrom-Json ~ przeksztalca json na obiekt w powershell'u
```

#response.data.id ~ wyciągamy id z naszych infoamcji które dostaliśmy, po wyłaniu pliku

#wysyłamy zapytanie o informacji naszej analizy wystanego pliku

#GET ~ rzqdamy informacji od st

```
$result= curl.exe --silent -X GET
"https://www.virustotal.com/api/v3/analyses/$(($response.data.id))" -H "x-apikey:
$api_key" | ConvertFrom-Json
```

#Wysyłamy zapytanie o nasz skan aż do poki nie bedzie gotowy

#sprawdzamy czy jest "completed" jak nie powtarzamy wszystko w petli

```
while ($result.data.attributes.status -ne "completed") {
```

```
    Write-Host "pending..."
```

```
    Start-Sleep 30 #darmowa wersja zapewnia 4 zapytania na minute
```

#Wysyłamy zapytanie o nasz skan

```
$result= curl.exe --silent -X GET
```

```
"https://www.virustotal.com/api/v3/analyses/$(($response.data.id))" -H "x-apikey:
$api_key" | ConvertFrom-Json
}
```

#sprawdzamy rezultaty

```
if ($result.data.attributes.stats.malicious -gt 0) {
```

```
    Write-Output "plik jest niebezpieczny"
```

```
} elseif ($result.data.attributes.stats.suspicious -gt 0) {
```

```
    Write-Output "plik jest podejrzany"
```

```
} else {
```

```
    Write-Output "plik jest bezpieczny"
```

```
}
```

```
}
```

#funkcja ktora pobiera api_key i zwraca czy raport przeszedł test w którym sprawdza plik EICAR

```
function test_raport {
    param (
        [String]$api_key
    )
}
```

```
#pobieramy plik EICAR do pliku
curl.exe --silent -X GET "https://secure.eicar.org/eicar.com" | Out-File
"testfile.txt"

#zapisujemy wynik raportu w zmiennej output
$output = get_raport "testfile.txt" $api_key

#usuwamy plik EICAR
Remove-Item -Path "testfile.txt" | Out-Null

#match ~ sprawdza czy w tekście występuje specyficzny ciąg znaków
if($output -match "plik jest niebezpieczny"){
    Write-Host "Test zakończony pomyślnie"
}else{
    Write-Host "Test zakończony porażką"
}
}

#input
$file = Read-Host -Prompt "podaj ścieżkę do pliku: "
$api_key = Read-Host -Prompt "podaj swój api key: "

#wypisujemy hash
Write-Host "Suma kontrolna pliku: $(get_hash $file)"
#generujemy raport
Write-Host "$(get_raport $file $api_key)"

#[TRZEBA ODKOMNETOWAĆ] żeby wykonać test
#test_raport $api_key
```